Linux

Introduzione

Linux è un sistema operativo **multiutente(permette di creare più utenti con i propri comando e dati sulla stessa macchina, inoltre più utenti possono usare e accedere la stessa versione della macchina)** dove differenti utenti possono avere accesso al sistema avendo i propri dati, i propri programmi e impostazioni completamente separate da quelle di altri utenti oltre ad avere la possibilità di accedere alle risorse del sistema simultaneamente.

- L'operazione di autenticazione dell'utente, tramite nome utente (username) e password, che permette l'accesso alle risorse del sistema è detta login.
- Ad ogni username assegnato dall'amministratore del sistema corrisponde uno user-id (UID) assegnato dal sistema.
  Se si ha un nome utente (es: Antonio) quell'utente viene assegnato a un User-ID, quindi viene gestito in base all'user Id e non al nome utente.
  Il login può essere eseguito in ambienti diversi, grafici o testuali da locale o da remoto. L'autenticazione verifica che l'utente abbia i requisiti per accedere al sistema o ad un suo servizio e metterlo in condizione di interagire con la macchina.

Utente root

L'utente root (superuser, administrator) : **è un utente privilegiato** .
Ad esso sono riservati i compiti di gestione e configurazione della macchina.
L'utente root ha poteri assoluti sul sistema.
Esso può:

- **aggiungere, eliminare e modificare account degli utenti**
- **installare e configurare servizi**
- **aggiungere e modificare il file system**
- **distruggere tutto con un solo comando**

Commandi di root

Per sfruttare il privilegio root esistono **2 comandi base:**

1. **Il comando `su` :**
   Il comando `su` (substitute user) permette di cambiare in una sessione di terminale.
   Se viene eseguito senza argomento, cambia all'utente root

```bash
su -
```

Dopo aver eseguito questo comando, ti verrà richiesta la password dell'utente root. Una volta inserita correttamente, la tua shell sarà quella dell'utente root

2. **Usare `sudo` :**
   Permette agli utenti autorizzati di eseguire comandi come root senza dover cambiare utente.

```bash
sudo comand
```

Gruppi e password

Su Linux, e non solo, gli utenti possono essere messi all'interno di gruppi per gestirli più facilmente

Gruppi

- **Ogni utente può far parte di uno o più gruppi, definiti dall'amministratore del sistema.**
- **Ogni gruppo è identificato da un group name associato ad un group-id (GID) numerico.**

Password

- **Ogni utente può avere (e in seguito modificare) una propria password.**
- **La password non viene visualizzata sullo schermo: è personale e non deve essere rivelata**

> ⚠ **ATTENZIONE !**
>
> Il sistema operativo Linux distingue tra lettere maiuscole e minuscole e pertanto:
>
> - username: admin

- username: Admin
- username: ADMIN
  NON sono lo stesso utente.

Come anche i seguenti file:

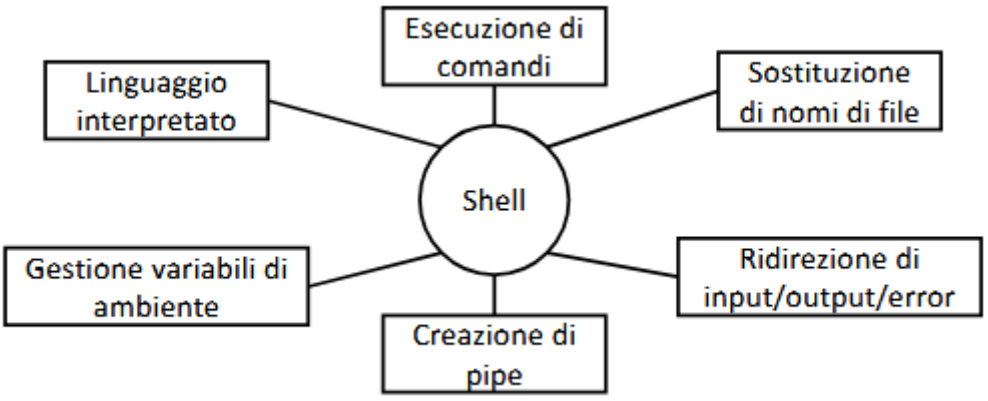
- prova.dat PROVA.dat
- Prova.dat PROVA.DAT
  NON rappresentano lo stesso file.

Shell

La shell (conchiglia): è un programma di sistema che agisce da interfaccia tra utente e il sistema operativo.
E' un interprete che legge ed esegue le istruzioni di comando per:
• **la creazione e l'avvio di processi.**
• **per la gestione di I/O, della memoria di massa e della memoria principale**.
• **per definire le politiche di protezione dei file e della comunicazione di rete**.
Quindi ogni volta che scriviamo un comando viene creato ed eseguita una shell che permette l'esecuzione di quel comando

> ⓘ **Info**
>
> Sebbene i sistemi Linux abbiano un'interfaccia utente grafica, la maggior parte dei programmatori e utenti esperti preferisce un'interfaccia a linea di comando, più veloce e potente, facilmente espandibile che permette all'utente di non dover sempre usare il mouse.

Quando la shell viene avviata:
• vi presenta il **prompt:**
spesso un simbolo di percentuale o di dollaro, che permette di inserire comandi.
• **interpreta il comando inserito**
• **crea e avvia il processo per la sua esecuzione**.
La shell è programmabile, cioè permette di definire gli script:
• **Uno script è un programma in formato testuale che racchiude comandi fondamentali per l'uso e l'amministrazione di un S.O**.



Shell.png

Tipi di Shell

Esistono vari tipi di shell: la shell originale dei sistemi Unix.

- Bourne shell (`sh`) : la shell originale dei sistemi Unix.
- Korn shell (`ksh`): più avanzata, con funzioni di scripting evolute.
- C shell (`csh` ed il suo successore `tcsh`): con sintassi ispirata al linguaggio C.
- Bourne again shell (bash): evoluzione della Bourne Shell, oggi la più diffusa.

Ci concentreremo su **bash:**

- la shell (quasi) standard in ambiente Linux.

ⓘ Differenza tra "Bash" e "Shell".

Nel linguaggio comune questi due termini vengono usati come sinonimi anche se in realtà non sono esattamente la stessa cosa.
Shell:
Il termine shell indica in generale l'interfaccia che permette all'utente di comunicare con il sistema operativo, tipicamente digitando comandi testuali.

> È quindi un **concetto generico**, non legato a un programma specifico.

Esistono diversi tipi di shell, ciascuno con proprie caratteristiche, sintassi e funzioni.

Vedi alcuni esempi di shell comuni in ambiente Unix/Linux

Bash:
Bash (acronimo di *Bourne Again SHell*) è **una particolare implementazione della shell**, sviluppata come miglioramento della Bourne Shell (`sh`).

È oggi la **shell predefinita nella maggior parte delle distribuzioni Linux** e su molte versioni di macOS.
Bash offre:

- **compatibilità con la vecchia `sh`;**
- **funzionalità estese (cronologia dei comandi, alias, scripting più avanzato, completamento automatico, ecc.);**
- **un ambiente interattivo molto pratico per gli utenti e gli sviluppatori.**

La possiamo quindi vedere cosi:

> **Tutti i sistemi che usano bash stanno usando una shell, ma non tutte le shell sono bash**

> **La "shell" è un termine generico che indica l'interfaccia (a riga di comando) tramite cui l'utente comunica con il sistema operativo. Mentre la Bash (Bourne Again Shell) è una specifica implementazione di shell, usata in molti sistemi Unix-like.**

Comandi di linux

I comandi rappresentano una richiesta di esecuzione.
Vanno scritti rispettando maiuscole, minuscole e spazi, altrimenti non verranno riconosciuti, cioè risulteranno "not found".

Forma generale di un comando:

```
comando [opzioni] [argomenti]
```

Opzioni:

- **modificano l'azione del comando.**
- iniziano (quasi sempre) con un `-`.

Argomenti

- **indicano l'oggetto su cui eseguire il comando.**

In sostanza sono gli input da dare ai comandi

Comando: `pwd`

Per verificare quale sia la directory corrente, si utilizza il comando `pwd` (**Print Working Directory**).

Stampa a video su terminale la directory corrente.

Es:

```shell
$pwd
```

Output:

```shell
/home/tua_home_directory
```

Comando: `passwd`

Permette di cambiare la password dell'utente.

Es:

```shell
$ passwd
```

Output:

```shell
Changing password for [current user]
Current password: [insert current password]
New password: [insert the new password]
Retype new password: [retype the new password]
Passwd: all authentication tokens updated successfully.
```

Comando: `man`

Permette di aprire il manuale (help on line) riguardante un comando specifico.

Es:

```shell
$ man passwd
```

Navigazione del manuale:

- `space` (barra spaziatrice): **va avanti di una pagina**
- `return` (invio): **va avanti di una linea**
- `q` : **esce dal file**

Nota

Utilizzare `$ sudo mandb` per aggiornare il database dei manuali.

`man -k`

Per ottenere le pagine di un manuale che contengono una determinata parola chiave si usa il comando `man -k` (dove `k` sta per keyword) seguito dalla parola chiave ricercata

Es:

```shell
$ man -k password
```

Da notare come `-k`, che è l'opzione del comando, cambia l'azione del comando.

Output:

```
Elenco di tutte le pagine manuale che contengono la parola chiave specificata.
```

Comando: `whatis`

Serve per ottenere una breve descrizione dei comandi di sistema si usa il comando `whatis`.
Es:

```shell
```

```
$ whatis pwd
```

Output:

```shell
pwd (1)    - print name of current/working directory
```

Oltre a questi comandi base visto sopra esistono moltissimi altri tipi di comandi in Linux, che fanno azioni specifiche in base al contesto.

Chi sono e chi è collegato

Questa serie di comandi mostrano gli utenti che sono collegati al sistema operativo e lo stanno utilizzando.

Il comando `$ whoami`:

Il comando per conoscere con quale identità si sta operando.

```bash
$ whoami
```

Il comando `$ id`:

Il comando per visualizzare lo UserID dell'utente corrente.

```bash
$ id
```

Il comando `$ id nome_utente`

Il comando per visualizzare lo UserID dell'utente specificato.

```bash
$ id nome_utente
```

Il comando `$ who`

Il comando per visualizzare tutti gli utenti loggati nel sistema.

```bash
$ who
```

Il comando `$ finger -l`

Serve per visualizzare le informazioni dell'utente corrente.

```bash
$ finger -l
```

> **Note**
>
> se il comando non viene trovato è possibile installarlo mediante il comando: `sudo apt install finger`

Il comando `$ finger nome_utente`

Il comando per visualizzare informazioni dell'utente specificato.

```bash
$ finger nome_utente
```

Il comando `$ last`

Il comando per visualizzare ed elencare gli ultimi utenti connessi.

```bash
$ last
```

Comando: `$ uname`

Per conoscere il nome del kernel del Sistema Operativo.

```bash
$ uname
```

Il comando `$ uname -a`:

Per visualizzare tutte le informazioni relative al Sistema Operativo in uso.

```bash
uname -a
```

Creazione di un file

Questa serie di comandi permette di creare, modificare e salvare i file.

Il comando `nano`

Il comando nano è un editor di testo a riga di comando semplice e facile da usare disponibile su molti sistemi Unix e Linux.
È particolarmente utile per modificare file di configurazione o creare e modificare file di testo direttamente dal terminale.

Es:

```shell
nano prova.txt
```

Sequenze di controlli utili

- `CTRL+X` : **per uscire dall'editor**.
- `CTRL + S` : **per salvare il file**.
- `CTRL+ INVIO` : **per confermare il salvataggio del file**.
- `CTRL+O` : **per salvare il file**.
- `CTRL + INVIO` : **per confermare il salvataggio del file**.

Il comando nano consente anche di modificare un file di testo esistente.
Es:

```bash
$ nano prova1.txt
```

Modificare il file di testo:

- Usare le frecce direzionali per spostarsi tra le righe.

Il comando `$ touch`

Il comando `$ touch filename`:

- **se il file non esiste, crea un file vuoto**.
- **se il file esiste, aggiorna la data e l'ora dell'ultimo accesso al file**.

Example

`$ touch prova2.txt` - opera su singolo file

`$ touch prova3.txt prova4.txt prova5.txt` - opera su file multipli

Visualizzare il contenuto di un file

Per visualizzare il contenuto di un file leggibile (non binario) si possono utilizzare i seguenti comandi:

- `cat`
- `more`
- `most`
- `tail`
- `head`

Il comando `$ cat`:

mostra il contenuto del file facendolo scorrere completamente sullo schermo.

Es:

Bash

```bash
$ cat prova1.txt
$ cat cenerentola.txt
```

Completamento automatico:

È un modo attraverso il quale la shell aiuta l'utente a completare un comando. Durante la digitazione di un nome o di un comando, premendo il tasto Tab, si chiede alla shell di provare a completarlo.

Example

Bash

```bash
$ cat cene[Tab] rentola
```

I tasti ↑ ↓ richiamano i comandi usati in precedenza

Comando: `$ more`

mostra il contenuto del file, una pagina per volta.

Es:

Bash

```bash
$ more cenerentola.txt
```

Navigazione:

- `space (barra spaziatrice)` : va avanti di una pagina
- `return (invio)` : va avanti di una linea
- `q` : esce dal file.

Comandi `$ tail`, `$ head`

- `$ tail` : **mostra di default le ultime 10 linee di un file**.
  Es:

Bash

```bash
$ tail cenerentola.txt
```

- `$ head` : **mostra di default le prime 10 linee di un file**.
  Es:

```bash
$ head cenerentola.txt
```

> **Per entrambi i comandi l'opzione `-n` mostra le prime o le ultime n linee di un file.**

Esempio:

```bash
$ head -1 cenerentola.txt
```

- **mostra la prima linea del file cenerentola**

Esempio:

```bash
$ tail -2 cenerentola.txt
```

- **mostra le ultime 2 linee del file cenerentola**

Comando `$ clear`

Il comando `$ clear` consente di «pulire» la console.
Es:

```shell
$ clear
```

Comando `$ ls`

Il comando `ls` (list directory) in Linux:
è uno dei comandi più utilizzati per visualizzare il contenuto di directory.
Esso elenca i file e le directory presenti nella directory corrente o in una directory specificata in ordine alfabetico.

Esempio di sintanssi:

```shell
$ ls [opzioni] [directory] %% mostra file/directory nella directory corrente %%

$ ls home  %% mostra file/directory nella directory %%

$ ls --color=never  %% mostra senza l'uso colori %%
```

Comando `ls -l`

Per ottenere informazioni più dettagliate sui file bisogna usare l'opzione `-l`

```bash
$ ls -l
```

In Linux esistono anche i file nascosti che iniziano con il carattere `.` :

- **file di configurazione che i programmi collocano nelle home-directory degli utenti per memorizzare le impostazioni utente.**
  Per visualizzare anche i file nascosi bisogna utilizzare l'opzione `-a`

Abstract

`$ ls -a` - mostra i file nascosti e non nascosti
`$ ls -al` - combina le opzioni –a e l'opzione -l

Lettura dei file nascosti

I comandi sono

```bash
$ cat .profile
```

```bash
$ cat .bash_profile
```

consentono di leggere i file nascosti chiamati `profile` e `bash_history`.

Altre opzioni per il comando `ls`

Esistono molte altre opzioni per il comando ls. Per esempio:
`$ ls -lt` : **mostra un elenco dettagliato dei file secondo la data dell'ultima modifica.**
`$ ls -lS` : **ordina i file dal più grande al più piccolo**
`$ ls -lr` : **ordina i file dal più piccolo al più grande**

Esempio:

- `$ ls -al etc` : **mostra tutti i file nascosti e non, contenuti nella directory, etc. in modo dettagliato**
- `$ ls -alS etc` : **come sopra ma ordinandoli dal più grande al più piccolo**

File System information

Esaminiamo più in dettaglio le informazioni ricevute dal comando `ls -l` :

Esempio

Bash

```bash
$ ls -l /home/marco-cascio
```

- rw- rw-r- - : 1 marco-cascio marco-cascio 11371 giu 25 17:26 prova1
d rwxr-xr-x: 2 marco-cascio marco-cascio 4096 giu 20 13:47 Documents
l rwxrwxrwwx: 1 marco-cascio marco-cascio 19 nov 29 13:22 prova1→/home/renata/prova1

Il primo carattere è:
- `-` : **se si tratta di un file.**
- `d` : **se si tratta di un directory.**
- `l` : **se si tratta di un link simbolico (puntatore)**.

Come possiamo notare dall'immagine sotto i successivi gruppi di tre caratteri rappresentano i permessi attribuiti al file rispettivamente per il proprietario

del file, per il gruppo e per il resto del mondo.

User (u)			Group (g)			Others (o)		
r	w	x	r	w	x	r	w	x

File System information.png

. Essi possono assumere i valori:

- `r` : **permesso di lettura**
- `w` : **permesso di scrittura**
- `x` : **permesso di esecuzione (per le directory rappresenta il permesso di attraversamento)**
- `-` : **nessun permesso.**

```
-rw- rw- r--    1  marco-cascio marco-cascio  11371  giu  25   17:26  prova1
drwx r-x r-x    2  marco-cascio marco-cascio  4096   giu  20   13:47  Documents
lrwx rwx rwx    1  marco-cascio marco-cascio  19     nov  29   13:22  prova1→/home/renata/prova1
```

File System information_2.png

Per Spiegare meglio questa cosa guardiamo questa immagine:

```
-rw- rw- r--    1  marco-cascio marco-cascio  11371  giu  25   17:26  prova1
drwx r-x r-x    2  marco-cascio marco-cascio  4096   giu  20   13:47  Documents
lrwx rwx rwx    1  marco-cascio marco-cascio  19     nov  29   13:22  prova1→/home/renata/prova1
```

Spiegazione FSI.png

- Il **primo carattere della prima colonna** indica se l'elemento è un file, una directory o un link simbolico.
- I successivi caratteri della prima colonna indicano i permessi
- la terza e la **quarta** colonna indicano l'utente e il **gruppo proprietario** del file.
- la 5° colonna rappresenta la dimensione in byte del file
- la 6° e la 7° colonna rappresentano la data e l'ora dell'ultima modifica del file.
- l'ultima colonna è il nome del file

> **in caso di collegamenti simbolici (link simbolico), → è la destinazione del collegamento**

Creazione di una directory

Questa serie di comandi servono per creare, cambiare, spostarsi tra le directory.

Il comando `mkdir`

Serve per creare una directory.

Shell

```shell
$ mkdir [nome_nuovo_directory]
```

Il comando crea la directory `nome_nuova_directory` all'interno della directory corrente, ma è possibile specificare il path desiderato.

Esempio per creare diverse directory:

Shell

```shell
$ mkdir mia  %% crea una directory chiamata mia nella directory corrente %%

$ mkdir new  %% crea una directory chiamata new nella directory corrente %%

$ mkdirtua disney prova  %% crea tre directory chiamate tua, disney e prova nella directory corrente %%
$ mkdirmia/sua  %% crea la directorysua all'interno della directory mia %%
```

Spostarsi tra le cartelle: Il comando `cd`

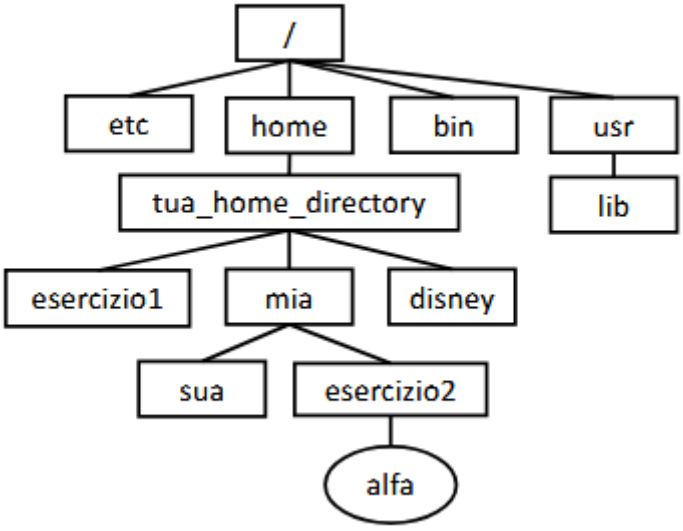
Serve per cambiare directory e muoversi nella gerarchia padre-figlio delle cartelle sul file system.

Bash

```bash
$ cd [nome_directory]
```

La directory specificata diviene la working directory.
Se nessuna directory viene specificata, si ritorna alla home directory.

Struttura del file system di linux



cd comand.png

Prendendo ad esempio questa immagine partiamo dalla directory `home` :

1. Se scriviamo

```shell
$ cd mia
```

Verremo mandati dentro la cartella `mia` ;
nello specifico il path sarà `home/tua_home_directory(o nome_user)/mia`
In aggiunto possiamo usare il comando `$ pwd` per controllare la directory nella quale ci troviamo.
Se volessi discendere ancora la gerarchia genitore-figlio delle directory:

```shell
$ cd sua
```

Adesso ci troviamo nella directory `sua` contenuta nella directory mia, il path è: `home/tua_home_directory/mia/sua` .

Per ritornare direttamente alla `tua_home_directory` :

```shell
$ cd
```

Questa semplice riga di comando ci riporta alla `tua_home_directory`

Spostamenti relativi e assoluti

Prima di continuare a vedere i prossimi comandi bisogna fare una distinzione tra gli spostamenti relativi e assoluti:

- Gli spostamenti o percorsi assoluti:
  Partono sempre dalla radice (cioè da `/`) e seguono l'intero 'indirizzo' della directory.
  Es:

```shell
$ cd /home/tua_home_directory/mia/esercizio2/alfa
```

Il vantaggio di questo percorso è che funziona da qualsiasi posizione nel file system.

- **Spostamenti o percorsi relativi:**
  Partono dalla working directory corrente (dove ti trovi).
  Es:
  Se ci si trova in `/home/tua_home_directory`

```shell
$ cd mia/disney
```

Ci si sposta in `/home/tua_home_directory/mia/disney`, allora in questo caso la directory `disney` diventa la working directory.

Se invece ci si trova in `/usr` e vuoi raggiungere `lib`:

```shell
$ cd lib
```

Ci si sposta in `/usr/lib` che diventerà la nostra working directory

✓ **Quando usare l'uno o l'altro**

Tipo	Quando usarlo	Esempio
Assoluto	Se devi raggiungere una cartella lontana o vuoi essere sicuro del percorso.	`cd /home/tua_home_directory`
Relativo	Per spostamenti rapidi tra directory vicine.	`cd mia/disney` (partendo da `tua_home_directory`)

Il comando `cd ./nome_directory`

Un altro modo per spostarsi discendo nella gerarchia padre-figlio delle directory è il comando `$ cd ./nome_directory` :

Es:

Shell

```
$ cd ./mia/sua
```

Ha lo stesso comportamento del comando

Shell

```
$ cd mia/sua
```

L'unica differenza è:

- **lo si usa per una maggiore leggibilità nel codice.**
  Quando si scrive

Shell

```
$ cd /mia/sua
```

il sistema assume automaticamente che si intenda `./mia/sua` .

Quando è utile usare questo tipo di comando?

1. Script e automazioni:

- Per evitare ambiguità, specialmente in script complessi dove è sempre meglio essere espliciti per migliorare la leggibilità del codice.

Shell

```
$ cd ./cartella_importante
```

2. Nomi ambigui:
   - Quando si ha una cartella nominata come un comando di sistema (es: `ls` o `test`)

Shell

```
$ cd ./test
```

In questo caso ci si sposta dentro la cartella `test`, anziché eseguire il comando `test`

3. Percorsi con spazi/nomi speciali:
   - Quando si hanno delle cartelle nominate con caratteri speciali e/o il nome contiene spazi.

Shell

```
$ cd "./ mia cartella"
```

Mentre invece è obbligatorio usare il `./` quando si vuole eseguire i file presenti nella directory corrente (senza `./`, il sistema cerca solo nei binari di sistema):

Shell

```
./mio_script.sh  # Senza `./` darà errore "comando non trovato"
```

Spostamento a ritroso nella directory

Oltre a discendere la gerarchia padre-figlio delle directory, la si può anche risalire andando a ritroso.
Per fare ciò esiste un comando specifico

Il comando `cd ..`

Questo comando permette di spostarsi dalla directory corrente a quella genitore.

Es:

Shell

```shell
mia-sua $ cd ...
mia $
```

In questo esempio ci trovavamo nella directory `sua`, ma con il comando `cd ..` siamo tornati nella directory genitore `mia`.

ⓘ Directory di Sistema – Percorsi assoluti

`$ cd/` :
- directory radice del sistema

`$ cd /bin:`
- contiene i programmi di sopravvivenza per il sistema

`$ cd /boot` :
- contiene i dati necessari all'avvio del sistema

`$ cd /dev` :
- contiene i cosiddetti devices

`$ cd /etc` :
- contiene i file di configurazione

`$ cd /lib` :
- contiene le librerie, ovvero parti di codice condivise tra i programmi

`$ cd /mnt` :
- contiene i mount-point

`$ cd /proc` :
- il mount-point del file system proc

`$ cd /root` :
- è la home directory dell'amministratore di sistema, utente root

`$ cd /sbin` :
- contiene programmi che solitamente richiedono i privilegi di root

`$ cd /tmp` :
- contiene i file temporanei

`$ cd /var` :
- contiene alcuni file in attesa di essere processati, come file di stampa

Verifica del tipo di file

Per verificare di che tipo sia un file (eseguibili, dati, directory) si può utilizzare il comando `file`.

Es:

Shell

```shell
$ file prova2.txt #Output: prova2.txt: ASCII text
```

Permessi sui file espressi in forma di stringa

Come abbiamo già visto sopra ad ogni file o directory in un sistema Linux sono associati permessi in base a tre categorie standard del sistema, che sono:

- **possessore del file stesso (u)**;
- **membri del suo gruppo (g)** ;
- **altri utenti (o)**;

I permessi vengono suddivisi, all'interno di queste tre categorie, in coppie di tre. Guardiamo questa immagine per capire meglio:

User (u)			Group (g)			Others (o)		
r	w	x	r	w	x	r	w	x

Permessi sui file.png

I permessi vengono suddivisi in coppie di tre per ogni categoria, e si suddividono in:

1. **Read(`r`)**: lettura.
   Permette di leggere il contenuto dei file e di leggere l'elenco dei file contenuti in una directory.
2. **Write(`w`):** Scrittura.
   Permette di modificare il contenuto di un file, e di aggiungere, rimuovere, rinominare file in una directory.
3. **Execute(x):** Esecuzione.
   Permette di eseguire un file e da la possibilità di attraversare una directory o accedervi tramite nome del percorso.

Il comando per vedere i permessi associati ad un file è `ls -l` :

```bash
$ ls -l pluto.txt #Output : - rw-rw-r -- 1 marco marco 26 apr 11:30 pluto.txt
```

Questo output - **rw-** **rw-** **r--** sta ad indicare che, questo file ha il permesso di

- **Lettura (r) e scrittura (w) per l'utente**, il quale non ha il permesso di esecuzione (-)
- **Lettura (r) e scrittura (w) per il gruppo di utenti presenti sul sistema**, i quali non hanno il permesso di esecuzione (-)
- **di sola lettura (r) per il resto del mondo**, per cui non è possibile la scrittura (-) e l'esecuzione (-) di tale file.

Definire i permessi

Per definire i permessi di un file su Linux bisogna usare il comando `$ chmod`.
La sintassi è:

```shell
$ chmod [ugoa] [+-=] [rwx] [nome_file]
```

Dove `[ugoa]` sta per:

- `u` : **user(proprietario)**
- `g` : **gruppo (group)**
- `o` : **altri utenti(other)**
- `a` : **tutti (all)**

mentre `[+-=]` indica:

- `+` : **aggiungi**
- `-` : **rimuovi**
- =: **assegna**

Per ultimo `[r w x]` indica:

- `r` : **read**
- `w` : **write**
- `e` : **execute**

Copiare un file: comando `cp`

Permette di copiare un file.
La sintassi è:

```shell
$ cp [file da copiare] [file copia] [nome percorso della copia - opzionale]
```

Es:

```shell
$ cp prova1.txt pluto3.txt
```

- **In questo caso crea il file `pluto3.txt` che è una copia del file `prova1.txt`.**

Con il comando `cp` si può copiare un file in una directory:

```shell
$ cp prova.txt prova3.txt mia
```

Crea una copia del file `prova.txt`, `prova3.txt` all'interno della directory `mia`.

```shell
$ cp *mia/sua
```

In questo caso copia tutti i file nella directory corrente dentro la directory `sua`

```shell
$ cp pl* mia/sua
```

copia tutti i file il cui nome inizia per pl nella directory `sua`.

Inoltre è possibile, con il comando `cp` copiare una directory:

la sintassi è

```shell
$ cp -r [dir1] [dir2]
```

Es. **se** `dir2` **non esiste:**

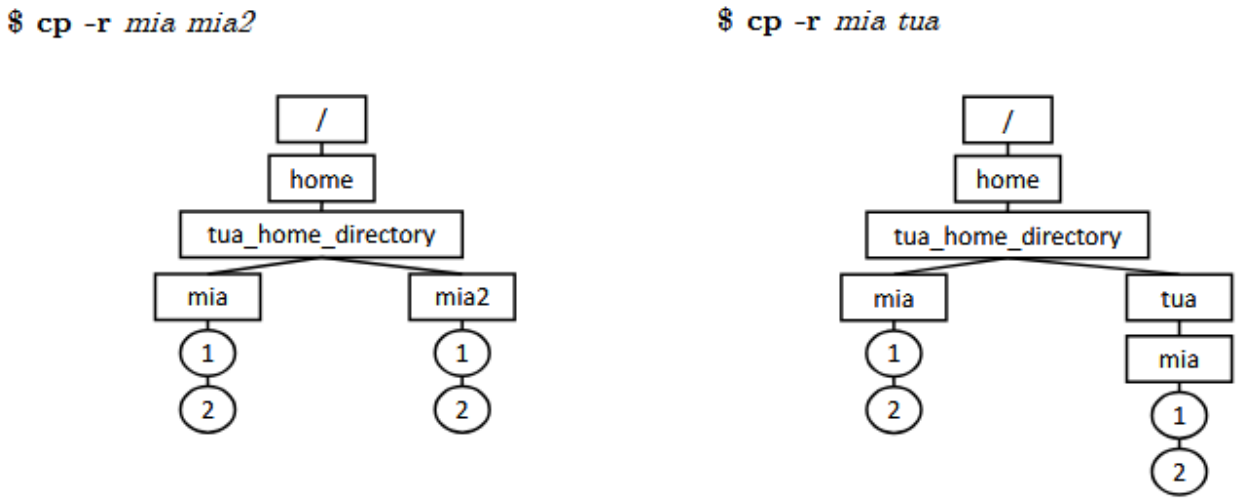
```shell
$ cp -r mia mia2
```

- **copia solo il contenuto della directory** `mia` **in una nuova directory** `mia2`**.**

Es. se `dir2` esiste:

```shell
$ cp -r mia tua
```

copia la directory `mia` **e il suo contenuto dentro la directory** `tua`**.**



cp comand.png

Eliminare i file: il comando `rm`

Questo comando permette di eliminare un file o una directory.
La sintassi è:

```shell
$ rm [nome_file] #Per eliminare un file
```

```
$ rm -r[nome_directory] #Per eliminare una directory
$ rmdir [nome_directory] #Per eliminare una directory vuota
$ rm -ri [nome_directory] #Per eliminare una directory e i suo contenuto chiedendo le conferme
```

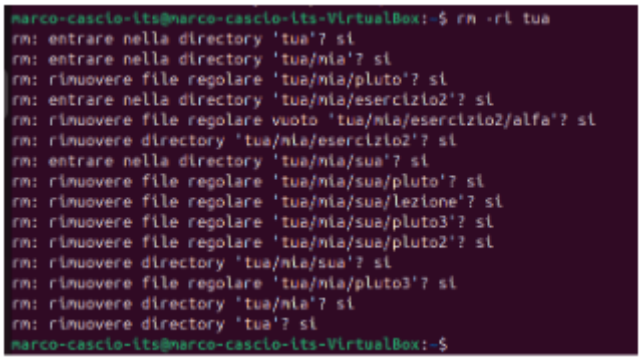
Es:

Shell

```
$ rm prova.txt  #elimina il file prova.txt

$ rm *  #elimina tutti i file nella directory corrente
$ rm pl*  #elimina tutti i file il cui nome inizia per pl

$ rmdir new  #elimina la directory new (solo se vuota)

$ rm -r mia2 #elimina la directory mia2 con tutto il suo contenuto
```

Shell

```
$ rm -ri tua
```



rm -ri.png

⚠ ATTENZIONE!!!!

Da utente privilegiato non eseguire MAI i comandi:

`$ rm -rf .*` :

- **cancella tutti i file e le directory nella working directory**

`$ rm -rf /` :

- **cancellerà tutti i file e le directory nel file system a partire dalla** `root` **`/`**

`$ rm -rf --no-preserve-root /` :
- **come sopra ma bypassando meccanismi di protezione**

Spostare e/o rinominare un file

Il comando `mv` (a.k.a. move):

> **permette di rinominare o spostare un file o una directory.**

Le opzioni di questo comando sono :

Shell

```
$ mv [file1] [file2] # rinomina il file1 in file2
$ mv [file1] [dir1/dir2] # sposta file1 dalla directory corrente alla directory dir1/dir2
$ mv [file1] [dir1/dir2/file2]
# sposta file1 nella directory dir1/dir2 cambiandogli nome in file2
```

Il comando `grep`

Questo comando **è utilizzato per la ricerca di stringhe all'interno di un file.**

Shell

```
Esempio: $ grep volta ./disney/cenerentola.txt
```

Ricerca la stringa volta all'interno del file `cenerentola.txt`, contenuto nella directory `/home/User/disney`.

Mentre per ricercare espressioni contenenti spazi o Tab si usano gli apici (`' '`).

Shell

```
$ grep ‘uomo ricco’ ./disney/cenerentola.txt
```

- Ricerca la stringa `'uomo ricco'` all'interno del file cenerentola.

L'opzione `-i`

Serve ad ignorare i caratteri maiuscoli o minuscoli.

Shell

```shell
$ grep -i pettinaci ./disney/cenerentola.txt
```

- Ricerca la stringa `'pettinaci'` all'interno del file, nel nostro caso questa parola inizia con la P maiuscola, ignorando se le lettere sono maiuscole o minuscole.

Commando `find`

Permette di cercare un file specifico all'interno del file system.

Bash

```bash
$ find [nomedir] [maxdepth] -name [nomefile]
```

La sintassi è:

- `[nomedir]` :
  - **è il percorso della directory da cui effettuare la ricerca**
- `[maxdepth]` :
  - **è il numero di livelli di directory in cui cercare**
- `[nomefile]` :
  - **è il nome del file da cercare.**

Se non mostra nulla vuol dire che non esiste nessun file in quella posizione.

Per cercare il file nelle directory si usa l'opzione `maxdepth` :

Bash

```bash
$ find . -maxdepth 1 -namepluto
```

questa opzione in questo caso cerca nel primo livello delle sub-directory, per cercare nei livelli successivi si usano i numeri `2` , `3` e cosi via.
Detto questo per trovare un file:

Bash

```bash
$ find . -name pluto
```

Cerca il file `pluto` nella working directory e nelle sue sotto directory.

L'opzione `-empty`

Cerca i file vuoti nella working directory e sub-directory.

```
$ find . -empty
```

L'opzione `-exec cat{}`

```bash
$ find . -name pluto -exec cat {} \;
```

cerca il file `pluto` ed esegue il comando `cat`

- `\;` : **chiude la sequenza di comandi da eseguire**

Il comando `tree`

Serve per esplorare in maniera grafica, ad albero, una directory specificata.

> ⓘ **Se il comando no viene trovato lo si può installare con il comando:** `sudo apt install tree`

```shell
$ tree -
```

- **Stampa l'albero della working directory**

```shell
$ tree [dir]
```

- **Stampa l'albero della directory** `dir` **specificata**

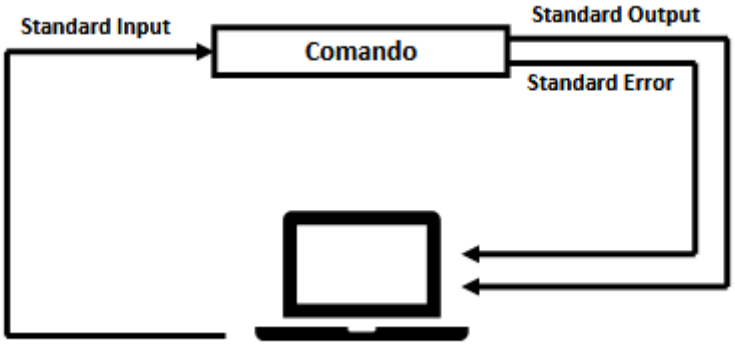
```shell
$ tree -d
```

- **Mostra solo le directory senza mostrare i file.**

Re-direzione dell' I/O

Quando un comando viene eseguito ha alcuni canali standard per il flusso dei dati:

- standard input: **dati di ingresso**
- standard output: **dati di uscita**
- standard error: **eventuali messaggi di errore.**
  La re-direzione è la deviazione dei canali standard di un dato comando verso destinazioni (o da sorgenti, nel caso dello standard input) che sono diverse da quelle predefinite.



Redirezione I-O.png

Standard input

Per re-direzionare lo standard si utilizza il simbolo `<`.
Il comando che precede il simbolo `<` considera come input il contenuto del file specificato subito dopo.

Esempio di casi d'uso

1. Inserire nella directory `/home/[tua_home_directory]` i file `sum.sh` e `number.txt`
2. Definire i permessi di esecuzione per il file `sum.sh`:
   Per fare ciò bisogna usare il comando `chmod`

Shell

```
$ chmod u+x sum.sh
```

In questo modo stiamo dando i permessi all'utente (`u`):

- `+` : **stiamo assegnando i permessi all'utente**
- `x` : **gli stiamo dando i permessi di esecuzione.**

3. Scrivere:

```shell
$./sum.sh < number.txt.
```

3. **Lo script viene eseguito prendendo in input il numero `10` contenuto nel file `number.txt` .**

Standard Output

Per re-direzionare lo standard output si utilizzano i simboli: `>` oppure `>>` . **L'output del comando che precede il simbolo `>` viene re-diretto nel file specificato subito dopo, creandolo o sovrascrivendolo** .

Esempi di casi d'uso.

```shell
$ ls > folder_list.txt
```

- **L'output del comando `ls` viene scritto nel nuovo file `folder_list.txt` .**
- **Se questo file non esiste lo crea o può sovrascrivere il suo contenuto se è presente**

```shell
$ echo 20 > number.txt
```

l'output del comando `echo` è stato scritto nel file esistente `number.txt` che viene sovrascritto.

```bash
```

```
$ echo "20" > number.txt
$ cat number.txt
#Output:
20
```

- Quindi il numero `20` viene aggiunto al file `number.txt` e sovrascrive il suo contenuto.

Mentre quando si vuole aggiungere contenuto senza sovrascrivere il contenuto già presente nel file, si usano i simboli `>>`.

Bash

```
$ echo 30 >> number.txt
#Output:
20
30
```

- Quindi il file `number.txt` contiene il numero `20`, scrivendo questa riga di comando si va ad aggiungere il numero 30 al file

Il comando `echo`.

Il comando `echo`:

- **viene utilizzato per visualizzare una linea di testo o una variabile nel terminale.**

Shell

```
$ echo Ciao Mondo!
```

- Stampa su terminale "Ciao Mondo!"

Shell

```
$ echo Ciao Mondo > mio_file.txt
```

- "Ciao Mondo" viene scritto all'interno di `mio_file.txt` sovrascrivendolo.

Shell

```
$ echo mi aggiungo al file! >> mio_file.txt
```

- Stessa cosa per la stringa `mi aggiungo al file!`

Pipeline

Nel linguaggio comune, una *pipeline* richiama l'idea di una **catena di montaggio:**

- **un processo composto da più fasi sequenziali, dove l'output di una fase diventa l'input della successiva.**
  Ad esempio, in un'azienda che sviluppa software, la realizzazione di un prodotto segue più passaggi — **progettazione**, **design** e **implementazione** — collegati tra loro in modo ordinato.

Nel **shell programming**, il concetto è analogo.

- **Una pipeline permette di collegare più comandi in sequenza, in modo che l'output di un comando diventi automaticamente l'input del successivo.**
  In questo modo è possibile combinare comandi semplici per ottenere elaborazioni più complesse in un'unica linea di codice.

Pipeline.png

Esempi:

1. Scrivere, utilizzando la pipe, il comando per visualizzare da un elenco esteso della directory `/home` tutti i file e le directory che nel loro nome contengono le lettere *li*.
   Per ovviare a questo problema si deve usare i comandi:

```bash
$ ls -al | grep li
```

2. Scrivere, utilizzando la pipe, il comando per cercare la parole *Cenerentola* nella prima riga del file `cenerentola.txt`

```shell
cat./disney/cenerentola.txt | head-1 | grep Cenerentola
```

Lista di commandi

Seguendo lo stesso principio delle **pipeline**, è possibile eseguire più comandi in sequenza all'interno di una **lista di comandi**, separandoli con il punto e virgola (`;`).

- **Una lista di comandi è dunque una sequenza composta da uno o più comandi, scritti sulla stessa riga e separati da `;` .**

Ogni comando viene eseguito **nell'ordine in cui appare**, indipendentemente dal successo o dal fallimento di quello precedente.

```shell
command1 ; command2 ; command3 ; …
```

Esempio

```
$ echo Buongiorno! ; echo La data di oggi! ; date ; cal
```

In questo caso il terminale:

1. stampa il messaggio *"Buongiorno!"*;
2. stampa il messaggio *"La data di oggi è:"*;
3. mostra la data corrente;
4. visualizza il calendario del mese in corso.

> **se il comando cal non viene trovato è possibile installarlo mediante `sudo apt install ncal`**

Le opzioni delle parentesi per le liste di comandi.

Le parentesi tonde `(command1 ; command2 ; command3)` nella shell vengono utilizzate per raggruppare comandi insieme in modo che vengano eseguiti in una sotto-shell:

- **si intende un ambiente separato che esegue i comandi come se fossero un singolo comando.**

Ad esempio:

```shell
$ echo Ciao ; echo Caro! > out_file.txt
```

Questa riga di comando esegue entrambi i comandi, ma:

- `"Ciao"` viene visualizzato su terminale,
- mentre l'output dell'ultimo `echo`: "Caro!" viene scritto in un file `out_file.txt`.
  Utilizzando le parentesi si sta indicando che entrambi gli output del comando `$echo` **devono essere scritti nel file** `out_file.txt`:

```shell
$ (echo Ciao ; echo ‘’ Caro!’’) > out_file.txt
```

Esegue entrambi i comandi e scrive l'output complessivo, formato dalle parole `"Ciao"` e `"Caro!"`, nel file `out_file.txt`.

Gli operatori logici

L'operatore AND (&&)

Questo operatore esegue il comando successivo solo se il comando precedente è stato eseguito con successo.

```shell
command1 && command2
```

- In questo caso `command2` viene eseguito solo se `command1` è stato eseguito con successo.

Esempio

Se `out_file.txt` **esiste**:

```shell
$ rm out_file.txt && echo il file è stato eliminato!
```

L'output sarà : *"Il file è stato eliminato"*

Questo perché il comando `$ rm out_file.txt` è stato eseguito con successo.

Mentre se il file **non esiste**:

```shell
$ rm
out_file.txt && echo il file è stato eliminato!
```

L'output sarà: *impossibile rimuovere file out_file.txt*

Questo perché il comando `rm` non è stato eseguito con successo.

L'operatore logico OR (||)

L'operatore `||` esegue il comando successivo solo se l'esecuzione del comando precedente è fallita.

```shell
command1 || command2
```

- `command2` **viene eseguito solo se** `command1` **è stato eseguito senza successo**

Se il file `out_text.txt` **non esiste**:

```shell
$ rm out_file.txt || echo il file non esiste!
```

L'output sarà: *"Il file non esiste"*
Questo perché non è stato possibile eseguire il comando `rm`